You

我想要照著圖片生成一個n x n 網格地圖 範圍為(5-9) 然後可以依序點選起點(綠色) 終點(紅色) 障礙物n-2個(灰色)顯示每個單元格的隨機生成行動（上下左右箭頭）作為策略。使用策略評估推導出每個狀態的價值 V(s)。分成vaule matrix 和 policy matrix

---

Gemini

這是一個結合了 **Flask (後端)**、**HTML/JavaScript (前端)** 以及 **強化學習 (Reinforcement Learning)** 基礎算法的有趣作業。

為了完成這個需求，我們需要實現 **策略評估 (Policy Evaluation)**。通常假設：

1. **獎勵 (Reward)**：到達終點為 +1（或更大），撞到障礙物或出界為 -1，每走一步為 -0.01。
2. **折扣因子 (γ)**：通常設為 0.9 或 1.0。
3. **狀態轉移**：在此作業中，策略是隨機生成的箭頭，代表每個格子有固定的行動。

以下是完整的實作架構：

---

1. Flask 後端 (`app.py`)

負責處理邏輯計算，特別是根據隨機策略計算 $V(s)$。

```
from flask import Flask, render_template, request, jsonify
import numpy as np

app = Flask(__name__)

def policy_evaluation(grid_size, start, end, obstacles, policy_matrix):
    # 初始化價值矩陣
    V = np.zeros((grid_size, grid_size))
    gamma = 0.9  # 折扣因子
    threshold = 1e-4

    # 定義行動對應的座標變化
    actions = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}

    while True:
        new_V = np.copy(V)
        delta = 0
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                action = policy_matrix[r][c]
                dr, dc = actions[action]
                nr, nc = r + dr, c + dc

                # 計算獎勵與下一狀態價值
                if 0 <= nr < grid_size and 0 <= nc < grid_size:
                    if [nr, nc] in obstacles:
                        reward = -1
                        next_val = V[r][c] # 撞牆留在原地
                    elif [nr, nc] == end:
                        reward = 10
                        next_val = 0 # 終止狀態
                    else:
                        reward = -0.1
                        next_val = V[nr, nc]
                else:
                    reward = -1 # 出界
                    next_val = V[r][c]

                v = reward + gamma * next_val
                new_V[r, c] = v
                delta = max(delta, abs(v - V[r, c]))

        V = new_V
        if delta < threshold:
            break
    return V.tolist()

@app.route('/')
```

```
def index():
    return render_template('index.html')

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    n = data['n']
    start = data['start']
    end = data['end']
    obstacles = data['obstacles']
    policy = data['policy']

    value_matrix = policy_evaluation(n, start, end, obstacles, policy)
    return jsonify({'value_matrix': value_matrix})

if __name__ == '__main__':
    app.run(debug=True)
```

---

2. 前端介面 (`templates/index.html`)

使用原生 JavaScript 處理點擊邏輯與表格生成。

```
<!DOCTYPE html>
<html>
<head>
    <title>Grid World HW</title>
    <style>
        .grid { display: grid; gap: 2px; margin-top: 20px; }
        .cell { width: 50px; height: 50px; border: 1px solid #ccc;
                display: flex; align-items: center; justify-content: center; cursor: pointer; font-size: 12px; }
        .start { background-color: #4CAF50 !important; color: white; }
        .end { background-color: #f44336 !important; color: white; }
        .obstacle { background-color: #808080 !important; color: white; }
        .matrix-container { display: flex; gap: 50px; }
        .label { font-weight: bold; margin-bottom: 10px; }
    </style>
</head>
<body>
    <h2>HW1 網格地圖開發</h2>
    <label>Enter n (5-9): </label>
    <input type="number" id="gridInput" min="5" max="9" value="5">
    <button onclick="generateGrid()">Generate Square</button>

    <p id="status">請依序點選：起點 (綠) -> 終點 (紅) -> 障礙物 (灰)</p>

    <div class="matrix-container">
        <div>
            <div class="label">Policy Matrix</div>
            <div id="policyGrid" class="grid"></div>
        </div>
        <div>
            <div class="label">Value Matrix</div>
            <div id="valueGrid" class="grid"></div>
        </div>
    </div>

    <script>
        let n, start = null, end = null, obstacles = [];
        let policy = [];

        function generateGrid() {
            n = parseInt(document.getElementById('gridInput').value);
            const container = document.getElementById('policyGrid');
            container.style.gridTemplateColumns = `repeat(${n}, 50px)`;
            container.innerHTML = '';
            document.getElementById('valueGrid').innerHTML = '';

            start = null; end = null; obstacles = [];
            policy = Array.from({ length: n }, () =>
                Array.from({ length: n }, () => ['U', 'D', 'L', 'R'][Math.floor(Math.random() * 4)])
            );

            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    const cell = document.createElement('div');
                    cell.className = 'cell';
                    cell.id = `cell-${r}-${c}`;
                    cell.innerText = policy[r][c].replace('U','↑').replace('D','↓').replace('L','←').replace('R','→');
                    cell.onclick = () => handleCellClick(r, c);
                    container.appendChild(cell);
                }
            }
        }

        function handleCellClick(r, c) {
            const cell = document.getElementById(`cell-${r}-${c}`);
            if (!start) {
                start = [r, c];
                cell.classList.add('start');
            } else if (!end) {
```

```
                if (r === start[0] && c === start[1]) return;
                end = [r, c];
                cell.classList.add('end');
            } else if (obstacles.length < n - 2) {
                if ((r === start[0] && c === start[1]) || (r === end[0] && c === end[1])) return;
                obstacles.push([r, c]);
                cell.classList.add('obstacle');
                if (obstacles.length === n - 2) calculateValues();
            }
        }

        async function calculateValues() {
            const response = await fetch('/calculate', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({ n, start, end, obstacles, policy })
            });
            const data = await response.json();
            renderValueGrid(data.value_matrix);
        }

        function renderValueGrid(matrix) {
            const container = document.getElementById('valueGrid');
            container.style.gridTemplateColumns = `repeat(${n}, 50px)`;
            container.innerHTML = '';
            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    const cell = document.createElement('div');
                    cell.className = 'cell';
                    if (r === start[0] && c === start[1]) cell.classList.add('start');
                    if (r === end[0] && c === end[1]) cell.classList.add('end');
                    if (obstacles.some(o => o[0] === r && o[1] === c)) cell.classList.add('obstacle');
                    cell.innerText = matrix[r][c].toFixed(2);
                    container.appendChild(cell);
                }
            }
        }
    </script>
</body>
</html>
```

關鍵功能說明：

1. **網格生成**：根據用戶輸入的 n 動態調整 CSS `grid-template-columns`。
2. **交互邏輯**：
    - 第一次點擊：設定為 **Start**（綠色）。
    - 第二次點擊：設定為 **End**（紅色）。
    - 後續點擊（直到 $n-2$ 次）：設定為 **Obstacles**（灰色）。
3. **隨機策略**：在網格初始化時，每個格子隨機分配 `['U', 'D', 'L', 'R']`，並在前端以箭頭顯示。
4. **價值評估**：
    - 後端 `policy_evaluation` 使用迭代法。
    - 根據貝爾曼方程（Bellman Equation）$V(s) = R + \gamma V(s')$ 更新。
    - 計算完成後，前端會並排顯示 **Policy Matrix**（帶箭頭）與 **Value Matrix**（帶數值）。

您需要將 `app.py` 運行起來，並在同級目錄下建立 `templates` 資料夾存放 HTML 檔案。請問需要我針對 **價值評估的公式** 做更深入的解釋嗎？

You

後端程式不能執行

Gemini

從截圖中可以看到，報錯訊息是 `ModuleNotFoundError: No module named 'flask'`。

這表示在您目前的虛擬環境 (`py38_env`) 中，還沒有安裝 **Flask** 這個套件。雖然環境已經激活，但每個虛擬環境的套件是獨立的，需要手動安裝。

請按照以下步驟解決：

1. 安裝 Flask

在您的 VS Code Terminal 中（確定左側還有顯示 (`py38_env`)），輸入以下指令並按 Enter：

```
pip install flask numpy
```

(我也加上了 numpy，因為計算 Value Matrix 會用到它)

---

2. 檢查安裝是否成功

安裝完成後，您可以輸入這條指令確認 Flask 是否已存在於該環境：

```
pip list
```

在列表中應該能看到 `Flask` 及其版本號。

---

3. 再次執行程式

確認安裝完成後，再次執行您的程式碼：

```
python DRL_HW_Gridworld.py
```

常見小提醒

- **檔案存放位置**：請確保您的 `index.html` 放在與 `DRL_HW_Gridworld.py` 同一層目錄下的 `templates` 資料夾內，結構如下：

  ```
  HW10309/
  ├── DRL_HW_Gridworld.py
  └── templates/
      └── index.html
  ```

- **執行成功後**：Terminal 會顯示 `Running on http://127.0.0.1:5000`，這時請打開瀏覽器輸入該網址就能看到網格地圖了。

需要我幫您檢查 HTML 檔案的擺放位置，或是程式碼中是否有其他拼字錯誤（例如截圖中出現過的 `render_templates` 多了個 s）嗎？

You

我的兩個matrix要相反過來放 然後請幫我多一個計算按鍵要和網格生成按鍵分開 按完計算按鍵才生成箭頭並計算 然後在幫我多加下一個步驟的提示

Gemini

沒問題！我已經根據您的需求調整了程式碼。

主要的修改如下：

1. **位置對調**：將 **Value Matrix** 放在左邊，**Policy Matrix** 放在右邊。
2. **按鈕分離**：
   - `Generate Square`：僅生成空白網格（只有數字），讓您點選起點、終點和障礙物。
   - `Calculate & Generate Policy`：點完後才隨機生成箭頭（策略）並計算價值。
3. **步驟提示**：新增一個提示區塊，會根據您的操作即時更新下一步該做什麼。

1. 前端代碼 (`templates/index.html`)

請更新您的 HTML 檔案內容：

```
<!DOCTYPE html>
<html>
<head>
    <title>強化學習網格地圖</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        .grid { display: grid; gap: 2px; margin-top: 10px; }
        .cell { width: 55px; height: 55px; border: 1px solid #ccc;
                display: flex; align-items: center; justify-content: center; cursor: pointer; font-size: 14px; background: white; }
        .start { background-color: #4CAF50 !important; color: white; }
        .end { background-color: #f44336 !important; color: white; }
        .obstacle { background-color: #808080 !important; color: white; }
        .matrix-container { display: flex; gap: 40px; margin-top: 20px; }
        .label { font-weight: bold; font-size: 18px; margin-bottom: 5px; color: #333; }
        .controls { margin-bottom: 20px; padding: 15px; background: #f9f9f9; border-radius: 8px; }
        #status-box { padding: 10px; background: #e3f2fd; border-left: 5px solid #2196f3; margin-bottom: 15px; font-weight: bold; }
        button { padding: 8px 15px; cursor: pointer; }
    </style>
</head>
<body>
    <h2>HW1 網格地圖開發與策略評估</h2>

    <div id="status-box">提示：請輸入地圖尺寸並點擊「Generate Square」</div>

    <div class="controls">
        <label>網格維度 n (5-9): </label>
        <input type="number" id="gridInput" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
        <button id="calcBtn" onclick="calculateValues()" disabled>Calculate & Generate Policy</button>
    </div>

    <div class="matrix-container">
        <div>
            <div class="label">Value Matrix</div>
            <div id="valueGrid" class="grid"></div>
        </div>
        <div>
            <div class="label">Policy Matrix</div>
            <div id="policyGrid" class="grid"></div>
        </div>
    </div>

    <script>
        let n, start = null, end = null, obstacles = [];
        let policy = [];
        const statusBox = document.getElementById('status-box');
```

```
const calcBtn = document.getElementById('calcBtn');

function updateStatus(msg) {
    statusBox.innerText = "提示：" + msg;
}

function generateGrid() {
    n = parseInt(document.getElementById('gridInput').value);
    if (n < 5 || n > 9) { alert("請輸入 5 到 9 之間的數字"); return; }

    const pContainer = document.getElementById('policyGrid');
    const vContainer = document.getElementById('valueGrid');

    pContainer.style.gridTemplateColumns = `repeat(${n}, 55px)`;
    vContainer.style.gridTemplateColumns = `repeat(${n}, 55px)`;

    pContainer.innerHTML = '';
    vContainer.innerHTML = '';

    start = null; end = null; obstacles = [];
    calcBtn.disabled = true;

    // 先生成 Policy Grid 作為交互界面
    for (let r = 0; r < n; r++) {
        for (let c = 0; c < n; c++) {
            const cell = document.createElement('div');
            cell.className = 'cell';
            cell.id = `p-cell-${r}-${c}`;
            // 初始顯示網格編號（如圖片所示）
            cell.innerText = (r * n + c + 1);
            cell.onclick = () => handleCellClick(r, c);
            pContainer.appendChild(cell);

            // 同步生成空的 Value Grid
            const vCell = document.createElement('div');
            vCell.className = 'cell';
            vCell.id = `v-cell-${r}-${c}`;
            vCell.innerText = (r * n + c + 1);
            vContainer.appendChild(vCell);
        }
    }
    updateStatus("請點選一個格子作為「起點」(綠色)");
}

function handleCellClick(r, c) {
    const pCell = document.getElementById(`p-cell-${r}-${c}`);
    const vCell = document.getElementById(`v-cell-${r}-${c}`);

    if (!start) {
        start = [r, c];
        pCell.classList.add('start');
        vCell.classList.add('start');
        updateStatus("請點選一個格子作為「終點」(紅色)");
    } else if (!end) {
        if (r === start[0] && c === start[1]) return;
        end = [r, c];
        pCell.classList.add('end');
        vCell.classList.add('end');
        updateStatus(`請點選 ${n-2} 個「障礙物」(灰色)，還剩 ${n-2} 個`);
    } else if (obstacles.length < n - 2) {
        if ((r === start[0] && c === start[1]) || (r === end[0] && c === end[1])) return;
        if (obstacles.some(o => o[0] === r && o[1] === c)) return;

        obstacles.push([r, c]);
        pCell.classList.add('obstacle');
        vCell.classList.add('obstacle');

        let remaining = (n - 2) - obstacles.length;
        if (remaining > 0) {
            updateStatus(`請繼續點選障礙物，還剩 ${remaining} 個`);
        } else {
            updateStatus("設定完成！請點擊「Calculate & Generate Policy」按鈕");
            calcBtn.disabled = false;
        }
    }
}

async function calculateValues() {
    // 在計算時才隨機生成 Policy
    policy = Array.from({ length: n }, () =>
        Array.from({ length: n }, () => ['U', 'D', 'L', 'R'][Math.floor(Math.random() * 4)])
    );

    updateStatus("正在計算價值矩陣...");

    const response = await fetch('/calculate', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({ n, start, end, obstacles, policy })
    });
```

```
            const data = await response.json();

            renderResults(data.value_matrix);
            updateStatus("計算完成！左側為價值矩陣 V(s)，右側為隨機策略矩陣。");
        }

        function renderResults(vMatrix) {
            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    // 更新 Policy 箭頭
                    const pCell = document.getElementById(`p-cell-${r}-${c}`);
                    if (!pCell.classList.contains('end') && !pCell.classList.contains('obstacle')) {
                        const arrow = policy[r][c].replace('U','↑').replace('D','↓').replace('L','←').replace('R','→');
                        pCell.innerText = arrow;
                    }

                    // 更新 Value 數值
                    const vCell = document.getElementById(`v-cell-${r}-${c}`);
                    if (vCell.classList.contains('obstacle')) {
                        vCell.innerText = ""; // 障礙物不顯示數值
                    } else {
                        vCell.innerText = vMatrix[r][c].toFixed(2);
                    }
                }
            }
        }
    </script>
</body>
</html>
```

2. 後端代碼 (DRL_HW_Gridworld.py)

請確保後端處理邏輯一致（修正了上次提到可能出現的拼字錯誤）：

```
from flask import Flask, render_template, request, jsonify
import numpy as np

app = Flask(__name__)

def policy_evaluation(grid_size, start, end, obstacles, policy_matrix):
    V = np.zeros((grid_size, grid_size))
    gamma = 0.9
    threshold = 1e-4
    actions = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}

    while True:
        new_V = np.copy(V)
        delta = 0
        for r in range(grid_size):
            for c in range(grid_size):
                # 終點與障礙物價值不更新
                if [r, c] == end or [r, c] in obstacles:
                    continue

                action = policy_matrix[r][c]
                dr, dc = actions[action]
                nr, nc = r + dr, c + dc

                # 邊界與獎勵判定
                if 0 <= nr < grid_size and 0 <= nc < grid_size:
                    if [nr, nc] in obstacles:
                        reward = -1
                        next_val = V[r][c]
                    elif [nr, nc] == end:
                        reward = 10
                        next_val = 0
                    else:
                        reward = -0.1
                        next_val = V[nr, nc]
                else:
                    reward = -1
                    next_val = V[r][c]

                v = reward + gamma * next_val
                new_V[r, c] = v
                delta = max(delta, abs(v - V[r, c]))

        V = new_V
        if delta < threshold:
            break
    return V.tolist()

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    value_matrix = policy_evaluation(
        data['n'], data['start'], data['end'], data['obstacles'], data['policy']
```

```
    )
    return jsonify({'value_matrix': value_matrix})

if __name__ == '__main__':
    app.run(debug=True)
```

操作說明：

1. **啟動程式**：在終端機輸入 `python DRL_HW_Gridworld.py`。
2. **生成地圖**：輸入數字後點點擊 `Generate Square`。這時畫面會出現兩排數字網格，且上方會提示「點選起點」。
3. **設定階段**：
    - 點第 1 下：起點（綠）。
    - 點第 2 下：終點（紅）。
    - 點後續幾下：障礙物（灰）。提示文字會告訴您還剩幾個障礙物要點。
4. **計算階段**：點滿障礙物後，`Calculate` 按鈕會解鎖。點擊它後，右邊網格會出現**箭頭**，左邊網格會出現**計算出的價值**。

You

我想要讓兩邊的matrix都可以做點選

Gemini

為了讓兩邊的 Matrix 都能夠捕捉點擊事件，我們需要將 `handleCellClick` 的邏輯同步到兩邊的對應單元格，並確保點擊任何一邊時，另一邊的樣式也會即時更新。

我已經調整了邏輯，現在無論您點擊左側（Value Matrix）還是右側（Policy Matrix），它們都會**同步更新狀態（顏色）**。

主要改動點：

1. **統一事件處理**：現在為兩邊的單元格都綁定了相同的 `handleCellClick`。
2. **狀態同步**：點擊其中一個單元格時，透過 ID 對應，同時對左右兩邊的 DOM 元素加上樣式類別（start, end, obstacle）。

修改後的 `templates/index.html` 部分片段：

請將 HTML 中的 `handleCellClick` 與生成網格的部分替換為以下代碼：

```
// 修改後的生成網格邏輯：兩邊都綁定點擊事件
function generateGrid() {
    n = parseInt(document.getElementById('gridInput').value);
    const pContainer = document.getElementById('policyGrid');
    const vContainer = document.getElementById('valueGrid');

    pContainer.innerHTML = '';
    vContainer.innerHTML = '';

    for (let r = 0; r < n; r++) {
        for (let c = 0; c < n; c++) {
            // 建立左側 Value Grid 單元格
            const vCell = document.createElement('div');
            vCell.className = 'cell';
            vCell.id = `v-cell-${r}-${c}`;
            vCell.innerText = (r * n + c + 1);
            vCell.onclick = () => handleCellClick(r, c);
            vContainer.appendChild(vCell);

            // 建立右側 Policy Grid 單元格
            const pCell = document.createElement('div');
            pCell.className = 'cell';
            pCell.id = `p-cell-${r}-${c}`;
            pCell.innerText = (r * n + c + 1);
            pCell.onclick = () => handleCellClick(r, c);
            pContainer.appendChild(pCell);
        }
    }
    // ... 後續邏輯保持不變
}

// 修改後的點擊處理：同時操作左右兩邊
function handleCellClick(r, c) {
    // 檢查目標格子是否已經被選取 (避免重複點擊)
    const pCell = document.getElementById(`p-cell-${r}-${c}`);
    if (pCell.classList.contains('start') || pCell.classList.contains('end') || pCell.classList.contains('obstacle')) return;

    if (!start) {
        start = [r, c];
        updateClass(r, c, 'start');
        updateStatus("請點選一個格子作為「終點」(紅色)");
    } else if (!end) {
        end = [r, c];
        updateClass(r, c, 'end');
        updateStatus(`請點選 ${n-2} 個「障礙物」(灰色)`);
    } else if (obstacles.length < n - 2) {
        obstacles.push([r, c]);
        updateClass(r, c, 'obstacle');

        let remaining = (n - 2) - obstacles.length;
```

```
        if (remaining > 0) {
            updateStatus(`還剩 ${remaining} 個障礙物`);
        } else {
            updateStatus("設定完成！請點擊「Calculate & Generate Policy」");
            calcBtn.disabled = false;
        }
    }
}

// 輔助函數：同時更新左右兩邊的樣式
function updateClass(r, c, className) {
    document.getElementById(`v-cell-${r}-${c}`).classList.add(className);
    document.getElementById(`p-cell-${r}-${c}`).classList.add(className);
}
```

這樣設計的好處：

- **操作靈活**：用戶可以在任何一邊點擊，不需要受限於必須在特定側點選。
- **視覺一致性**：因為我們透過 `updateClass` 函數同時對兩個 DOM 操作，所以當您在 Value Matrix 點下障礙物時，Policy Matrix 的對應位置也會自動變灰，防止錯誤操作。

這樣修改後，您的介面將會非常直觀。如果您在執行時遇到任何點擊不同步的情況，請檢查 `v-cell` 和 `p-cell` 的 ID 是否在 `generateGrid` 中正確產生。

是否需要我為您加入一個「重置 (Reset)」按鈕，讓您可以快速清空當前地圖重新設定？

You

請將程式改玩丟回來給我

Gemini

沒問題！我已經將所有的邏輯整合好，並修正了讓兩邊的 Matrix（價值與策略）都可以點選的功能。這份程式碼已經包含了完整的 `app.py` 和 `index.html`，你可以直接替換舊檔案。

1. 後端程式 (`DRL_HW_Gridworld.py`)

請確保此檔案與 `templates/` 資料夾在同一個目錄下。

```
from flask import Flask, render_template, request, jsonify
import numpy as np

app = Flask(__name__)

def policy_evaluation(grid_size, start, end, obstacles, policy_matrix):
    V = np.zeros((grid_size, grid_size))
    gamma = 0.9
    threshold = 1e-4
    actions = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}

    while True:
        new_V = np.copy(V)
        delta = 0
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                action = policy_matrix[r][c]
                dr, dc = actions[action]
                nr, nc = r + dr, c + dc

                if 0 <= nr < grid_size and 0 <= nc < grid_size:
                    if [nr, nc] in obstacles:
                        reward = -1
                        next_val = V[r][c]
                    elif [nr, nc] == end:
                        reward = 10
                        next_val = 0
                    else:
                        reward = -0.1
                        next_val = V[nr, nc]
                else:
                    reward = -1
                    next_val = V[r][c]

                v = reward + gamma * next_val
                new_V[r, c] = v
                delta = max(delta, abs(v - V[r, c]))

        V = new_V
        if delta < threshold:
            break
    return V.tolist()

@app.route('/')
def index():
    return render_template('index.html')
```

```
@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    value_matrix = policy_evaluation(
        data['n'], data['start'], data['end'], data['obstacles'], data['policy']
    )
    return jsonify({'value_matrix': value_matrix})

if __name__ == '__main__':
    app.run(debug=True)
```

---

2. 前端介面 (templates/index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>強化學習網格地圖</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        .grid { display: grid; gap: 2px; }
        .cell { width: 50px; height: 50px; border: 1px solid #999;
                display: flex; align-items: center; justify-content: center; cursor: pointer; font-size: 14px; background: white; }
        .start { background-color: #4CAF50 !important; color: white; }
        .end { background-color: #f44336 !important; color: white; }
        .obstacle { background-color: #808080 !important; color: white; }
        .matrix-container { display: flex; gap: 50px; margin-top: 20px; }
        .label { font-weight: bold; margin-bottom: 10px; }
        #status-box { padding: 15px; background: #e3f2fd; border-left: 5px solid #2196f3; margin-bottom: 20px; }
    </style>
</head>
<body>
    <h2>網格地圖開發 (n x n)</h2>
    <div id="status-box">請設定網格尺寸並點擊「Generate Square」。</div>

    <div>
        <input type="number" id="gridInput" min="5" max="9" value="5">
        <button onclick="generateGrid()">Generate Square</button>
        <button id="calcBtn" onclick="calculateValues()" disabled>Calculate & Generate Policy</button>
    </div>

    <div class="matrix-container">
        <div>
            <div class="label">Value Matrix</div>
            <div id="valueGrid" class="grid"></div>
        </div>
        <div>
            <div class="label">Policy Matrix</div>
            <div id="policyGrid" class="grid"></div>
        </div>
    </div>

    <script>
        let n, start = null, end = null, obstacles = [], policy = [];
        const calcBtn = document.getElementById('calcBtn');

        function generateGrid() {
            n = parseInt(document.getElementById('gridInput').value);
            const pGrid = document.getElementById('policyGrid');
            const vGrid = document.getElementById('valueGrid');
            pGrid.style.gridTemplateColumns = vGrid.style.gridTemplateColumns = `repeat(${n}, 50px)`;
            pGrid.innerHTML = vGrid.innerHTML = '';
            start = null; end = null; obstacles = []; calcBtn.disabled = true;

            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    ['v', 'p'].forEach(type => {
                        const cell = document.createElement('div');
                        cell.className = 'cell';
                        cell.id = `${type}-cell-${r}-${c}`;
                        cell.innerText = (r * n + c + 1);
                        cell.onclick = () => handleCellClick(r, c);
                        type === 'v' ? vGrid.appendChild(cell) : pGrid.appendChild(cell);
                    });
                }
            }
            document.getElementById('status-box').innerText = "請點選「起點」(綠色)。";
        }

        function handleCellClick(r, c) {
            const id = `v-cell-${r}-${c}`;
            if (document.getElementById(id).classList.contains('start') ||
                document.getElementById(id).classList.contains('end') ||
                document.getElementById(id).classList.contains('obstacle')) return;

            if (!start) {
                start = [r, c];
                updateClasses(r, c, 'start');
                document.getElementById('status-box').innerText = "請點選「終點」(紅色)。";
            } else if (!end) {
```

```
                end = [r, c];
                updateClasses(r, c, 'end');
                document.getElementById('status-box').innerText = `請點選 ${n-2} 個「障礙物」(灰色)。`;
            } else if (obstacles.length < n - 2) {
                obstacles.push([r, c]);
                updateClasses(r, c, 'obstacle');
                if (obstacles.length === n - 2) {
                    document.getElementById('status-box').innerText = "設定完成！請點選「Calculate」開始計算。";
                    calcBtn.disabled = false;
                }
            }
        }

        function updateClasses(r, c, cls) {
            document.getElementById(`v-cell-${r}-${c}`).classList.add(cls);
            document.getElementById(`p-cell-${r}-${c}`).classList.add(cls);
        }

        async function calculateValues() {
            policy = Array.from({ length: n }, () => Array.from({ length: n }, () => ['U', 'D', 'L', 'R'][Math.floor(Math.random() * 4)])
            const res = await fetch('/calculate', { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({
            const data = await res.json();

            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    const pCell = document.getElementById(`p-cell-${r}-${c}`);
                    const vCell = document.getElementById(`v-cell-${r}-${c}`);
                    if (!pCell.classList.contains('start') && !pCell.classList.contains('end') && !pCell.classList.contains('obstacle'))
                        pCell.innerText = {'U':'↑','D':'↓','L':'←','R':'→'}[policy[r][c]];
                        vCell.innerText = data.value_matrix[r][c].toFixed(2);
                    }
                }
            }
            document.getElementById('status-box').innerText = "計算完成。";
        }
    </script>
</body>
</html>
```

這份程式碼已經滿足了你所有的要求：雙向同步點擊、按鈕分開、步驟提示，以及左右矩陣互換。需要我進一步解釋其中的 `policy_evaluation` 演算法邏輯嗎？

You

你可以幫我看他算對嗎?

Gemini

從您的截圖來看，目前的計算邏輯有幾個明顯的狀況，我們可以一起分析一下：

1. 關於價值評估的合理性

在強化學習的 **策略評估 (Policy Evaluation)** 中，價值函數 $V(s)$ 是代表從該狀態開始，遵循當前策略所能獲得的「預期累積獎勵」。

- **異常點**：您 Value Matrix 中出現了正數（如左上角的 `1`），以及某些地方呈現 `-10.00`。
- **可能性分析**：
  - **終止狀態 (Terminal State)**：在您的程式碼中，`end` 點應該具有很高的價值（例如 +10 或 +100），而其他普通格子應該逐步向它靠近。
  - **獎勵設定**：如果您的獎勵設定（Reward Function）在到達終點時沒有給予足夠的獎勵，或者計算公式中對於「撞到邊界/障礙物」的懲罰過重（例如 `-10`），數值就會出現劇烈波動。

2. 演算法檢查 (`DRL_HW_Gridworld.py`)

請檢查您的 `policy_evaluation` 函數中這一段邏輯：

```
elif [nr, nc] == end:
    reward = 10
    next_val = 0 # 這裡是關鍵，到達終點後價值應為 0 或終點獎勵
```

如果您發現計算出來的值看起來很奇怪，請確認 **折扣因子 (γ)** 是否正確設定（通常為 0.9 或 0.99）。

3. 如何驗證正確性

我們可以透過「**貝爾曼期望方程式 (Bellman Expectation Equation)**」來進行手動驗證： $$V(s) = \sum_{a} \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$$

對於您的地圖：

1. **策略一致性**：檢查一下 `Policy Matrix` 中箭頭指向的格子，其對應的 `Value Matrix` 中的數值是否符合該方向的預期（例如：指向一個高價值格子的箭頭，該格子本身的價值應該也較高）。
2. **邊界處理**：如果箭頭指向邊界，您的程式邏輯是 `reward = -1` 且 `next_val = V[r][c]`（原地不動）。如果這部分的懲罰過大，數值就會一直往負數累積。

建議嘗試：

您可以試著將獎勵邏輯稍微調整，觀察數值變化：

- **終點 (End)**：設定為 `+10`
- **撞牆/障礙物 (Obstacle)**：設定為 `-1`

- **普通移動 (Step)**：設定為 `-0.1`

如果調整後數值依然呈現這樣的大幅跳動，請檢查 `while True` 循環中的 `delta` 收斂條件，確認程式碼是否真的跑完了完整迭代。

您想讓我幫您修改 `reward` 函數的邏輯，讓它產出的數值更符合一般強化學習教材（如 Sutton & Barto）的範例嗎？

You

那妳可以幫我改成找出最佳路徑嗎?

Gemini

要將「策略評估 (Policy Evaluation)」改為「找出最佳路徑」，我們需要將演算法從單純的評估隨機策略，升級為 **「策略迭代 (Policy Iteration)」** 或 **「價值迭代 (Value Iteration)」**。

這代表程式不再使用隨機的箭頭，而是會自動計算出每個格子**「最好的行動 (Optimal Action)」**，並更新 Policy Matrix。

以下是針對後端邏輯的關鍵調整，將其改為 **Value Iteration**（價值迭代），這通常是最快找到最佳策略的方法：

1. 修改後端 (`DRL_HW_Gridworld.py`)

請將原有的 `policy_evaluation` 函數替換為 `value_iteration`：

```
def value_iteration(grid_size, start, end, obstacles):
    V = np.zeros((grid_size, grid_size))
    policy = np.full((grid_size, grid_size), 'U') # 預設策略
    gamma = 0.9
    threshold = 1e-4
    actions = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}

    # 1. 價值迭代
    while True:
        delta = 0
        new_V = np.copy(V)
        for r in range(grid_size):
            for c in range(grid_size):
                if [r, c] == end or [r, c] in obstacles:
                    continue

                # 計算每個動作的價值，選出最大值
                action_values = []
                for a_name, (dr, dc) in actions.items():
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < grid_size and 0 <= nc < grid_size:
                        if [nr, nc] in obstacles:
                            reward = -1; next_v = V[r, c] # 撞牆留在原地
                        elif [nr, nc] == end:
                            reward = 10; next_v = 0
                        else:
                            reward = -0.1; next_v = V[nr, nc]
                    else:
                        reward = -1; next_v = V[r, c]
                    action_values.append(reward + gamma * next_v)

                new_V[r, c] = max(action_values)
                delta = max(delta, abs(new_V[r, c] - V[r, c]))
        V = new_V
        if delta < threshold: break

    # 2. 根據收斂後的 V 矩陣提取最佳策略
    for r in range(grid_size):
        for c in range(grid_size):
            if [r, c] == end or [r, c] in obstacles: continue
            best_a = None; max_v = -float('inf')
            for a_name, (dr, dc) in actions.items():
                nr, nc = r + dr, c + dc
                # 計算該行動帶來的價值，邏輯需與上面一致
                if 0 <= nr < grid_size and 0 <= nc < grid_size and [nr, nc] not in obstacles:
                    val = (10 if [nr, nc] == end else -0.1) + gamma * V[nr, nc]
                else:
                    val = -1 + gamma * V[r, c]
                if val > max_v: max_v = val; best_a = a_name
            policy[r, c] = best_a

    return V.tolist(), policy.tolist()

# 修改 app.route('/calculate')
@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    v_matrix, p_matrix = value_iteration(data['n'], data['start'], data['end'], data['obstacles'])
    return jsonify({'value_matrix': v_matrix, 'policy_matrix': p_matrix})
```

2. 修改前端 (`templates/index.html`)

你需要修改 `calculateValues` 函數，讓它能接收後端傳回來的最佳策略矩陣：

```
async function calculateValues() {
    updateStatus("正在計算最佳路徑...");
    const response = await fetch('/calculate', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({ n, start, end, obstacles }) // 不再需要傳送舊 policy
    });
    const data = await response.json();

    // 更新介面
    for (let r = 0; r < n; r++) {
        for (let c = 0; c < n; c++) {
            const pCell = document.getElementById(`p-cell-${r}-${c}`);
            const vCell = document.getElementById(`v-cell-${r}-${c}`);

            // 只有非終點、非障礙物才顯示箭頭
            if (!pCell.classList.contains('start') && !pCell.classList.contains('end') && !pCell.classList.contains('obstacle')) {
                const arrow = {'U':'↑','D':'↓','L':'←','R':'→'}[data.policy_matrix[r][c]];
                pCell.innerText = arrow;
                vCell.innerText = data.value_matrix[r][c].toFixed(2);
            }
        }
    }
    updateStatus("計算完成！這是最佳路徑策略 (Optimal Policy)。");
}
```

這次改動的核心邏輯：

1. **放棄隨機策略**：程式不再使用 `Math.random()` 決定箭頭方向。
2. **動態規劃 (Value Iteration)**：程式會自動嘗試「上、下、左、右」四個方向，比較哪一個方向能得到最高的回報 (`max(action_values)`)，並將該方向更新到 `Policy Matrix`。
3. **自動優化**：當您按下按鈕，系統會經過多次迭代，最終算出所有格子通往終點的「最佳路徑」。

這樣修改後，您的 Policy Matrix 就不會再出現亂跳的箭頭，而是會出現一條指向終點的明確路徑！您可以試試看重新執行，結果應該會變得非常有規律。

You

我要按下計算就直接產出 請改完全部程式回傳給我

Gemini

為了實現「按下計算按鈕即自動算出最佳路徑並顯示」的功能，我將後端改為 **Value Iteration (價值迭代法)**，它會根據您設定的起點、終點和障礙物，自動尋找每一格的最優策略。

請將以下兩個檔案內容分別替換到您的目錄中。

1. 後端程式 (`DRL_HW_Gridworld.py`)

這段程式使用動態規劃演算法來計算價值矩陣 V 與最佳策略矩陣 π。

```
from flask import Flask, render_template, request, jsonify
import numpy as np

app = Flask(__name__)

def value_iteration(n, start, end, obstacles):
    # 初始化 V 矩陣
    V = np.zeros((n, n))
    policy = np.full((n, n), 'U')
    gamma = 0.9
    actions = {'U': (-1, 0), 'D': (1, 0), 'L': (0, -1), 'R': (0, 1)}

    # 迭代計算 V(s)
    for _ in range(100): # 迭代 100 次以收斂
        new_V = np.copy(V)
        for r in range(n):
            for c in range(n):
                if [r, c] == end or [r, c] in obstacles: continue

                vals = []
                for dr, dc in actions.values():
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < n and 0 <= nc < n:
                        if [nr, nc] in obstacles: vals.append(-1 + gamma * V[r, c])
                        elif [nr, nc] == end: vals.append(10 + gamma * 0)
                        else: vals.append(-0.1 + gamma * V[nr, nc])
                    else: vals.append(-1 + gamma * V[r, c])
                new_V[r, c] = max(vals)
        V = new_V

    # 根據最終 V 計算最佳策略
    for r in range(n):
        for c in range(n):
            if [r, c] == end or [r, c] in obstacles: continue
            best_a = 'U'; max_v = -float('inf')
            for name, (dr, dc) in actions.items():
```

```
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < n and [nr, nc] not in obstacles:
                    v = (10 if [nr, nc] == end else -0.1) + gamma * V[nr, nc]
                else: v = -1 + gamma * V[r, c]
                if v > max_v: max_v = v; best_a = name
            policy[r, c] = best_a
    return V.tolist(), policy.tolist()

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.json
    v, p = value_iteration(data['n'], data['start'], data['end'], data['obstacles'])
    return jsonify({'v': v, 'p': p})

if __name__ == '__main__':
    app.run(debug=True)
```

2. 前端介面 (templates/index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>最佳路徑網格計算</title>
    <style>
        .grid { display: grid; gap: 2px; }
        .cell { width: 50px; height: 50px; border: 1px solid #999; display: flex; align-items: center; justify-content: center; cursor: p
        .start { background: #4CAF50; color: white; }
        .end { background: #f44336; color: white; }
        .obstacle { background: #808080; color: white; }
        .container { display: flex; gap: 40px; margin-top: 20px; }
    </style>
</head>
<body>
    <h2>網格最佳路徑計算</h2>
    <input type="number" id="n" value="5" min="5" max="9">
    <button onclick="gen()">Generate</button>
    <button onclick="calc()">Calculate Optimal Path</button>

    <div class="container">
        <div><div class="label">Value Matrix</div><div id="vGrid" class="grid"></div></div>
        <div><div class="label">Policy Matrix</div><div id="pGrid" class="grid"></div></div>
    </div>

    <script>
        let n, start, end, obs = [];
        function gen() {
            n = parseInt(document.getElementById('n').value);
            start = end = null; obs = [];
            ['vGrid', 'pGrid'].forEach(id => {
                let el = document.getElementById(id);
                el.style.gridTemplateColumns = `repeat(${n}, 50px)`;
                el.innerHTML = '';
                for(let i=0; i<n*n; i++) {
                    let c = document.createElement('div');
                    c.className = 'cell';
                    c.id = `${id}-${Math.floor(i/n)}-${i%n}`;
                    c.onclick = () => click(Math.floor(i/n), i%n);
                    el.appendChild(c);
                }
            });
        }
        function click(r, c) {
            if(!start) { start=[r,c]; update(r,c,'start'); }
            else if(!end) { end=[r,c]; update(r,c,'end'); }
            else if(obs.length < n-2) { obs.push([r,c]); update(r,c,'obstacle'); }
        }
        function update(r,c,cls) {
            document.getElementById(`vGrid-${r}-${c}`).classList.add(cls);
            document.getElementById(`pGrid-${r}-${c}`).classList.add(cls);
        }
        async function calc() {
            let res = await fetch('/calculate', {method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.stringify({n, sta
            let data = await res.json();
            for(let r=0; r<n; r++) {
                for(let c=0; c<n; c++) {
                    if(!document.getElementById(`vGrid-${r}-${c}`).classList.value.match(/start|end|obstacle/)) {
                        document.getElementById(`vGrid-${r}-${c}`).innerText = data.v[r][c].toFixed(2);
                        document.getElementById(`pGrid-${r}-${c}`).innerText = {'U':'↑','D':'↓','L':'←','R':'→'}[data.p[r][c]];
                    }
                }
            }
        }
    </script>
</body>
</html>
```

修改重點：

1. **自動化**：按下 `Calculate Optimal Path` 後，程式會自動根據 Value Iteration 演算法推導出全局最優解。
2. **邏輯收斂**：移除隨機性，保證每次計算出的箭頭都是指向目標的最佳路徑。
3. **直接產出**：前端接收到後端計算完畢的矩陣後，會一次性更新兩個矩陣顯示結果。

請在終端機重新啟動 `python DRL_HW_Gridworld.py` 即可測試。是否需要我協助說明其中的收斂邏輯？

You

我一樣需要提示下一步

Gemini

好的，沒問題！為了讓操作體驗更直觀，我為您加上了「操作狀態提示框 (Status Box)」，它會自動告知您下一步該點選什麼，直到完成設定並計算為止。

以下是優化後的完整前端程式 (`templates/index.html`)，請直接替換原本的檔案即可：

優化後的前端代碼 (`templates/index.html`)

```
<!DOCTYPE html>
<html>
<head>
    <title>最佳路徑網格計算</title>
    <style>
        body { font-family: sans-serif; padding: 20px; }
        .grid { display: grid; gap: 2px; }
        .cell { width: 50px; height: 50px; border: 1px solid #999; display: flex; align-items: center; justify-content: center; cursor: p
        .start { background: #4CAF50 !important; color: white; }
        .end { background: #f44336 !important; color: white; }
        .obstacle { background: #808080 !important; color: white; }
        .container { display: flex; gap: 40px; margin-top: 20px; }
        #status-box { padding: 15px; background: #e3f2fd; border-left: 5px solid #2196f3; margin-bottom: 20px; font-weight: bold; }
    </style>
</head>
<body>
    <h2>網格最佳路徑計算</h2>

    <div id="status-box">請設定網格尺寸並點擊「Generate」開始。</div>

    <div>
        <input type="number" id="n" value="5" min="5" max="9">
        <button onclick="gen()">Generate</button>
        <button id="calcBtn" onclick="calc()" disabled>Calculate Optimal Path</button>
    </div>

    <div class="container">
        <div><div class="label">Value Matrix</div><div id="vGrid" class="grid"></div></div>
        <div><div class="label">Policy Matrix</div><div id="pGrid" class="grid"></div></div>
    </div>

    <script>
        let n, start, end, obs = [];
        const statusBox = document.getElementById('status-box');
        const calcBtn = document.getElementById('calcBtn');

        function gen() {
            n = parseInt(document.getElementById('n').value);
            start = end = null; obs = [];
            calcBtn.disabled = true;
            statusBox.innerText = "請點選一個格子作為「起點」(綠色)。";

            ['vGrid', 'pGrid'].forEach(id => {
                let el = document.getElementById(id);
                el.style.gridTemplateColumns = `repeat(${n}, 50px)`;
                el.innerHTML = '';
                for(let i=0; i<n*n; i++) {
                    let c = document.createElement('div');
                    c.className = 'cell';
                    c.id = `${id}-${Math.floor(i/n)}-${i%n}`;
                    c.innerText = i + 1; // 顯示編號
                    c.onclick = () => click(Math.floor(i/n), i%n);
                    el.appendChild(c);
                }
            });
        }

        function click(r, c) {
            if(!start) { start=[r,c]; update(r,c,'start'); statusBox.innerText = "請點選一個格子作為「終點」(紅色)。"; }
            else if(!end) {
                if(r===start[0] && c===start[1]) return;
                end=[r,c]; update(r,c,'end');
                statusBox.innerText = `請點選 ${n-2} 個「障礙物」(灰色)。`;
            }
            else if(obs.length < n-2) {
                if((r===start[0] && c===start[1]) || (r===end[0] && c===end[1])) return;
                obs.push([r,c]); update(r,c,'obstacle');
                let rem = (n-2) - obs.length;
                if(rem > 0) statusBox.innerText = `還剩 ${rem} 個障礙物，請繼續點選。`;
```

```
                else { statusBox.innerText = "設定完成！請按下「Calculate Optimal Path」按鈕。"; calcBtn.disabled = false; }
            }
        }

        function update(r,c,cls) {
            document.getElementById(`vGrid-${r}-${c}`).classList.add(cls);
            document.getElementById(`pGrid-${r}-${c}`).classList.add(cls);
        }

        async function calc() {
            statusBox.innerText = "計算中，請稍候...";
            let res = await fetch('/calculate', {method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.stringify({n, sta
            let data = await res.json();
            for(let r=0; r<n; r++) {
                for(let c=0; c<n; c++) {
                    let vCell = document.getElementById(`vGrid-${r}-${c}`);
                    let pCell = document.getElementById(`pGrid-${r}-${c}`);
                    if(!vCell.classList.contains('start') && !vCell.classList.contains('end') && !vCell.classList.contains('obstacle')) {
                        vCell.innerText = data.v[r][c].toFixed(2);
                        pCell.innerText = {'U':'↑','D':'↓','L':'←','R':'→'}[data.p[r][c]];
                    }
                }
            }
            statusBox.innerText = "計算完成！已產生最佳路徑策略。";
        }
    </script>
</body>
</html>
```

這次的改進：

1. **動態狀態提示**：`status-box` 會根據您目前點選的步驟（起點 -> 終點 -> 障礙物）即時更新，完全解決了操作不知所措的問題。
2. **按鈕鎖定機制**：在障礙物點滿之前，`Calculate` 按鈕會呈現灰色禁用狀態，防止在設定未完成時就進行計算。
3. **視覺指引**：保留了之前的網格編號，讓地圖看起來更像課堂作業的標準格式。

現在您可以重新執行後端程式，並打開瀏覽器進行測試了！如果還有其他細節需要調整（例如字體大小或顏色），隨時告訴我。